

Was noch fehlt

Nur die offenen Punkte, in der Reihenfolge, in der sie zu tun sind · Stand 4. Oktober 2026

Der Server ist fertig und läuft. Tabellen, Auslöser, Edge Function und drei Cron-Jobs stehen und sind nachgemessen; der Minutenlauf kommt mit 200 am Gateway vorbei, und das Dienstkonto holt einen OAuth-Token. Alles, was hier steht, hängt an deinen Konten.

Die offenen Punkte

#	WAS	DRINGLICHKEIT	WO
1	APNs-Schlüssel bei Firebase prüfen — greift nicht	hier hängt es	Firebase + Apple
2	Apple-Mitgliedschaft im Team HACJC6623Z	wohl erledigt	Xcode
3	Push-Berechtigung für die App-ID app.matchup.mobile	wohl erledigt	developer.apple.com
4	Den .p8 -Schlüssel ersetzen (Kopien liegen in WhatsApp)	bewusst nein	entschieden
5	Datenschutzseite veröffentlichen	erledigt	25.09.2026
6	App-Datenschutz ausfüllen	vor Release	App Store Connect
7	Datensicherheit ausfüllen	vor Release	Play Console
8	Am echten iPhone prüfen	Beweis	Gerät + DB

Punkt 1 ist der einzige echte Blocker. Am 04.10.2026 gemessen: Ein Testpush an das registrierte iPhone scheitert mit Invalid APNs credential. Alles andere steht oder ist sehr wahrscheinlich erledigt; 6 und 7 brauchst du erst vor dem Store-Release.

1 APNs-Schlüssel bei Firebase prüfen

HIER HÄNGT ES · 5 MIN

Am 04.10.2026 gemessen: Ein Testpush an das seit dem 25.09. registrierte iOS-Gerät scheitert mit `HTTP 401 · Invalid APNs credential (THIRD_PARTY_AUTH_ERROR)`.

Was das genau heißt: FCM hat den Geräte-Token **angenommen** — sonst käme `UNREGISTERED`. Es ist dann mit Apple ins Gespräch gegangen, und **Apple hat die Zugangsdaten abgelehnt**. Der Fehler sitzt zwischen Firebase und Apple: nicht am Gerät, nicht in der App, nicht am Server.

Die Datei auf dem Mac ist in Ordnung (ebenfalls am 04.10. geprüft): `~/keys/AuthKey_9JF35BFS9Q.p8` ist ein gültiger privater Schlüssel, 256 Bit, PEM-Format, 257 Bytes — genau wie ein APNs-Auth-Key aussehen muss.

1. **Firebase-Konsole** → Zahnrad → **Projekteinstellungen** → **Cloud Messaging** → **Apple-App-Konfiguration**: Steht dort überhaupt ein Schlüssel?
2. Wenn ja: Lautet seine **Key ID** `9JF35BFS9Q` ? Es gab einen Vorgänger (`8887U4P9BH`) — ist der eingetragen oder die alte Datei hochgeladen, erklärt das den Fehler genau so.
3. Steht als **Team ID** `HACJC6623Z` da? Ein Tippfehler wirkt identisch.
4. **developer.apple.com** → **Keys**: Ist `9JF35BFS9Q` dort noch **aktiv** oder auf *Revoked*? Ein widerrufener Schlüssel wird abgelehnt, auch wenn die Datei gültig bleibt.

Fertig, wenn der Testpush durchgeht. Sag Bescheid — ich schicke ihn und lese die Antwort vor.

1 Apple-Mitgliedschaft auf diesem Mac

PFLICHT · 10 MIN

Warum: Ohne bezahltes Team darf die App keine Push-Benachrichtigungen verwenden, und es gibt kein Distribution-Zertifikat — derselbe Grund, aus dem `flutter build ipa` scheitert. **Zum Testen auf dem eigenen iPhone reicht ein Entwicklungs-Build;** das Distribution-Zertifikat brauchst du erst für TestFlight und den Store. Die bezahlte Mitgliedschaft brauchst du trotzdem, weil nur sie die Push-Berechtigung an der App-ID freischaltet.

1. Xcode öffnen.
2. Menü **Xcode** → **Settings...** (oder `⌘ + ,`).
3. Reiter **Accounts**.
4. Unten links auf **+** → **Apple ID** → **Continue**.
5. Mit der Apple ID anmelden, die zum **bezahlten** Team `HACJC6623Z` gehört.
6. Links das Konto anklicken. Rechts muss **MatchUp** (`HACJC6623Z`) stehen — Rolle *Account Holder*, *Admin* oder *App Manager*.
7. Rechts unten **Download Manual Profiles**.

```
security find-identity -v -p codesigning
```

Fertig, wenn dort eine Zeile mit *Apple Distribution* steht. Steht weiterhin nur *Apple Development*, bist du im falschen Team angemeldet oder die Mitgliedschaft ist abgelaufen — das ist der Stand von heute.

2 Push für die App-ID einschalten

PFLICHT · 3 MIN

Warum: Apple muss wissen, dass genau diese App Push benutzen darf. Ohne das Häkchen trägt das Provisioning-Profil kein `aps-environment`, und APNs weist jede Nachricht ab.

1. `developer.apple.com/account` öffnen, anmelden.
2. Oben rechts prüfen: Steht dort **MatchUp / HACJC6623Z**? Sonst umschalten — mit dem persönlichen Team gibt es die Option gar nicht.
3. Links **Certificates, Identifiers & Profiles** → **Identifiers**.
4. In der Liste `app.matchup.mobile` anklicken.
5. Häkchen bei **Push Notifications** setzen.
6. Oben rechts **Save**, Rückfrage mit **Confirm**.
7. **Danach in Xcode** noch einmal **Download Manual Profiles** — sonst baut Xcode weiter gegen das alte Profil ohne Push, und der Fehler ist später schwer zu finden.

Fertig, wenn neben *Push Notifications* ein Häkchen steht und die Seite *Enabled* anzeigt.

3 Den Schlüssel ersetzen

ENTSCHIEDEN · BLEIBT

Entschieden am 25.09.2026: Der Schlüssel wird nicht ersetzt. Damit gilt weiterhin die Key ID 9JF35BFS9Q . Der Abschnitt bleibt als Anleitung stehen — und als Notiz, dass das Risiko unten bekannt und bewusst getragen ist.

Der Grund: Von AuthKey_9JF35BFS9Q.p8 liegen Kopien in WhatsApp-Containern auf diesem Mac. **Wer diese Datei hat, kann Push-Nachrichten in deinem Namen verschicken** — an alle Geräte, die die App installiert haben. Die Key ID ist harmlos, die Datei ist das Geheimnis.

1. `developer.apple.com/account` → **Keys**.
2. Den Schlüssel 9JF35BFS9Q anklicken → **Revoke** → bestätigen.
3. + → **Key Name** MatchUp Push → Häkchen bei **Apple Push Notifications service (APNs)** → **Continue** → **Register** → **Download**.
4. **Die neue Key ID notieren** und bei Firebase eintragen (sie steht im Dateinamen).

```
mkdir -p ~/keys && mv ~/Downloads/AuthKey_*.p8 ~/keys/  
find ~ -name "AuthKey_*.p8" 2>/dev/null # alte Kopien aufspüren
```

6 App Store Connect → App-Datenschutz

VOR RELEASE · 10 MIN

Warum: Push erhebt genau **eine** zusätzliche Datenart — die Geräte-Kennung. Kein Tracking, keine Werbung. Genau das trägst du ein.

FRAGE	ANTWORT
Erfasst die App Daten?	Ja (war schon vorher so)
Zusätzlicher Datentyp durch Push	Identifikatoren → Geräte-ID
Verwendungszweck	App-Funktionalität
Mit der Identität des Nutzers verknüpft?	Ja — der Token hängt am Konto
Wird für Tracking verwendet?	Nein

Warum: Dieselbe Auskunft, andere Formulare. Google fragt zusätzlich nach Verschlüsselung und Löschung — beides ist gegeben.

FRAGE	ANTWORT
Zusätzlicher Datentyp	Geräte- oder andere IDs
Erhoben?	Ja
Geteilt?	Nein — Google ist Auftragsverarbeiter
Zweck	App-Funktionalität
Pflicht oder optional?	Optional — der Nutzer kann Push ablehnen
Bei Übertragung verschlüsselt?	Ja
Löschung auf Anfrage möglich?	Ja

Warum: Im Simulator ist Push mit Firebase unzuverlässig — ein Fehlschlag dort beweist nichts. Android ginge auch im Emulator. **Dieser Schritt ist zugleich der einzige echte Beweis für den APNs-Schlüssel** aus Punkt 4: Von außen lässt sich sein Status nicht abfragen.

1. App aufs iPhone bringen (geht erst nach Schritt 1): `flutter run --release -d <iphone>` oder über TestFlight.
2. App starten, die Nachfrage nach Benachrichtigungen **erlauben**.
3. Mit einem **zweiten Konto** eine Direktnachricht an dich schicken.

```
-- hat das Gerät sich gemeldet?  
select count(*) from push_geraete;  
  
-- legt der Auslöser an, und geht es raus?  
select id, art, gesendet_at, fehler from push_auftraege order by id desc limit 5;
```

Fertig, wenn `push_geraete` mindestens eine Zeile hat, binnen Sekunden ein Auftrag erscheint und er binnen einer Minute ein `gesendet_at` trägt — und das Telefon klingelt.

Wenn nichts ankommt, in dieser Reihenfolge:

- `push_geraete` leer → der Token wurde nie gemeldet. Berechtigung abgelehnt, oder Firebase ist auf dem Gerät nicht hochgekommen.
- Auftrag da, aber kein `gesendet_at` → die Spalte `fehler` derselben Zeile lesen.
- `fehler` nennt APNs oder *Auth* → Schritt 2 fehlt, oder der Schlüssel aus Punkt 4 liegt doch nicht richtig.